Queues at Work — Scheduling, BFS & The Big Picture

Lecture 08

Data Structures and Algorithms

Part 1: Job Scheduling with Queues

What is Job Scheduling?

Operating systems must decide **which process to run next** when multiple processes compete for the CPU.

Analogy: Air traffic control — planes must take off and land in an orderly fashion.

The simplest approach: **FIFO scheduling** — first process to arrive gets the CPU first.

```
Ready Queue:
  [P1] → [P2] → [P3] → [P4] → CPU

Processes are served in the order they arrive — just like a regular queue!
```

FCFS (First Come First Served) Scheduling

Use a **simple queue**: processes enqueue on arrival, dequeue when CPU is free.

Example

Process	Arrival Time	Burst Time (CPU needed)
P1	0	4
P2	1	3
P3	2	1

Gantt Chart:

```
| --- P1 --- | --- P2 --- | P3 |
0            4            7    8
```

Process	Start	Finish	Waiting Time	Turnaround Time
P1	0	4	0	4

The Convoy Effect

Problem with FCFS: A long process **blocks** all shorter ones behind it.

```
What if P1 has burst time = 20?

| -------- P1 (20 units) -------- | P2 (3) | P3 (1) |
0                                  20       23       24

P2 waits 19 units, P3 waits 21 units — terrible!
```

Short processes are stuck behind a long one — like being stuck behind a truck on a single-lane road.

Solution? Give each process a **limited time slice** and rotate — **Round Robin**!

Round Robin Scheduling — Queues in Action

Each process gets a fixed **time quantum** (e.g., 2 units). If not finished, it goes to the **rear of the queue**.

Uses a **circular queue** — a perfect fit!

Example (Time Quantum = 2)

Process	Arrival Time	Burst Time
P1	0	4
P2	1	3
P3	2	1

Round Robin: Step-by-Step Trace

```
Time 0-2:   P1 runs (remaining: 2)    Queue after: [P2, P3, P1]
Time 2-4:   P2 runs (remaining: 1)    Queue after: [P3, P1, P2]
Time 4-5:   P3 runs (remaining: 0) ✓  Queue after: [P1, P2]
Time 5-7:   P1 runs (remaining: 0) ✓  Queue after: [P2]
Time 7-8:   P2 runs (remaining: 0) ✓  Queue after: []    DONE!
```

Gantt Chart:

```
| P1 | P2 | P3| P1 | P2|
0    2    4   5    7   8
```

Process	Finish	Turnaround	Waiting
P1	7	7	3
P2	8	7	4
P3	5	3	2

Avg Waiting = (3 + 4 + 2) / 3 = 3.0 — Fairer than FCFS for mixed workloads!

Part 2: Breadth-First Search (BFS) and Queues

What is Breadth-First Search?

A **graph traversal** algorithm that explores all neighbors at the current depth before going deeper.

Analogy: Ripples in a pond — the wave expands **outward in all directions equally**.

Another analogy: Searching a building **floor by floor** (vs. DFS which takes the stairs deep into one wing).

```
Consider this graph:          BFS from A:

    A --- B --- E             Level 0: A
    |     |                   Level 1: B, C  (A's neighbors)
    C --- D --- F             Level 2: D, E  (B's & C's neighbors)
                              Level 3: F     (D's neighbor)

BFS Order: A → B → C → D → E → F
```

BFS naturally uses a **Queue**!

BFS Algorithm Using a Queue

```
BFS(startNode):
    create empty Queue Q
    enqueue startNode into Q
    mark startNode as visited

    while Q is not empty:
        node = dequeue(Q)
        process node
        for each UNVISITED neighbor of node:
            mark neighbor as visited
            enqueue neighbor into Q
```

Key insight: The queue ensures we process nodes in the order they were **discovered** — level by level, expanding outward.

BFS Trace Example

```
Graph:  A -- B -- E          BFS from A:
        |    |
        C -- D -- F

Step 1: Enqueue A, mark visited   Queue: [A]       Visited: {A}
Step 2: Dequeue A, process A      Queue: []         Visited: {A}
        Enqueue neighbors B, C    Queue: [B, C]     Visited: {A, B, C}
Step 3: Dequeue B, process B      Queue: [C]        Visited: {A, B, C}
        Enqueue neighbors D, E    Queue: [C, D, E]  Visited: {A, B, C, D, E}
Step 4: Dequeue C, process C      Queue: [D, E]     Visited: {A, B, C, D, E}
        (D already visited)       Queue: [D, E]
Step 5: Dequeue D, process D      Queue: [E]        Visited: {A, B, C, D, E}
        Enqueue neighbor F        Queue: [E, F]     Visited: {A, B, C, D, E, F}
Step 6: Dequeue E, process E      Queue: [F]
Step 7: Dequeue F, process F      Queue: []         DONE!

BFS Order: A → B → C → D → E → F
```

DFS (Stack) vs BFS (Queue) — Side by Side

Using the **same graph** from both Lecture 07 and this lecture:

Feature	DFS (Stack)	BFS (Queue)
Data Structure	Stack	Queue
Strategy	Go **deep** first	Go **wide** first
Traversal Order	Follows one path deeply	Explores level by level
Shortest Path?	✗ Not guaranteed	✓ Yes (unweighted graphs)
Memory	O(h) — h = max depth	O(w) — w = max width
Use Cases	Cycle detection, topological sort, maze solving	Shortest path, level-order traversal, social networks

Same Code, Different Data Structure

```
DFS(start):
    S = new Stack()
    S.push(start)

    while S is not empty:
        node = S.pop()
        if node not visited:
            visit node
            for each neighbor:
                S.push(neighbor)
```

```
BFS(start):
    Q = new Queue()
    Q.enqueue(start)
    mark start visited
    while Q is not empty:
        node = Q.dequeue()
        process node
        for each unvisited
            neighbor of node:
            mark visited
            Q.enqueue(neighbor)
```

Stack → depth-first exploration (most recent first)

Queue → breadth-first exploration (oldest first)

This is one of the most elegant ideas in computer science!

Part 3: More Applications of Stacks and Queues

Stack Application: Balanced Parentheses Checking

Problem: Given a string of brackets, check if they are properly nested.

Algorithm:

1. Scan left to right
2. **Opening bracket** `(`, `{`, `[` → **Push** onto stack
3. **Closing bracket** `)`, `}`, `]` → **Pop** from stack, check if it matches
4. At end: stack should be **empty** → balanced!

```
✓  {[()]}  →  Push {  Push [  Push (  Pop→( matches )
                                     Pop→[ matches ]
                                     Pop→{ matches }  Stack empty ✓

✗  {[(])}  →  Push {  Push [  Push (  Pop→( vs ] → MISMATCH! ✗

✗  {[()    →  Push {  Push [  Push (  Pop→( matches )
                                     Pop→[ matches ]
                                     Stack NOT empty (has {) → unbalanced ✗
```

Stack Application: Function Call Management

Every function call uses the **call stack** to manage execution:

```
main() calls funcA(), which calls funcB():

Call Stack:
┌──────────┐
│  funcB()  │  ← currently executing (TOP)
├──────────┤
│  funcA()  │  ← waiting for funcB to return
├──────────┤
│  main()   │  ← waiting for funcA to return
└──────────┘

Each stack frame stores:
  • Local variables
  • Return address (where to resume)
  • Parameters

When funcB() returns → POP its frame → resume funcA()
When funcA() returns → POP its frame → resume main()
```

More Stack & Queue Applications

Stack Applications

- **String Reversal:** Push all characters, pop them → reversed string
- **Undo/Redo in editors:** Each action pushed; undo = pop
- **Browser Back button:** Pages pushed on visit; back = pop
- **Syntax parsing in compilers:** Matching tokens and scopes

Queue Applications

- **Print Spooling:** Documents queue up for the printer (FIFO)
- **Keyboard Buffer:** Keystrokes queued when typing faster than processing
- **Interrupt Handling:** Events queued for processing in real-time systems
- **Cache (LRU):** Least Recently Used page replacement uses a queue
- **Simulations:** Customers at a bank, cars at a toll booth

Part 4: Stacks & Queues — Unit Wrap-up

Journey Recap: The Full Picture

```
Lecture 04: Stack — LIFO, Array & Linked List implementations
Lecture 05: Queue — FIFO, Array & Linked List implementations
Lecture 06: Circular Queue, Deque, Priority Queue
Lecture 07: Stack Applications — Expressions, DFS
Lecture 08: Queue Applications — Scheduling, BFS
```

The Big Picture — How Everything Connects

```
ADTs (from previous unit)
  ├── Stack ADT (LIFO)
  │     ├── Array Implementation
  │     ├── Linked List Implementation
  │     └── Applications: Expressions, DFS, Parentheses, Recursion
  └── Queue ADT (FIFO)
        ├── Array Implementation (Linear → Circular)
        ├── Linked List Implementation
        ├── Variations: Deque, Priority Queue
        └── Applications: Scheduling, BFS, Buffering
```

Key Complexity Summary

Data Structure	Push/Enqueue	Pop/Dequeue	Peek/Front	Space
Stack (Array)	O(1)	O(1)	O(1)	O(n)
Stack (Linked List)	O(1)	O(1)	O(1)	O(n)
Queue (Array)	O(1)	O(1)	O(1)	O(n)
Queue (Linked List)	O(1)	O(1)	O(1)	O(n)
Circular Queue	O(1)	O(1)	O(1)	O(n)
Deque (Doubly LL)	O(1)	O(1)	O(1)	O(n)
Priority Queue (Heap)	O(log n)	O(log n)	O(1)	O(n)

Key takeaway: All standard Stack and Queue operations are **O(1)** — that's what makes them so powerful!

Stack vs Queue — The Final Comparison

Feature	Stack	Queue
Principle	LIFO — Last In, First Out	FIFO — First In, First Out
Access Points	One end (top)	Two ends (front & rear)
Insert	push (at top)	enqueue (at rear)
Delete	pop (from top)	dequeue (from front)
Analogy	Stack of plates	Line at a counter
Graph Traversal	DFS (go deep)	BFS (go wide)
Scheduling	Call stack (recursion)	Job queue (FCFS, Round Robin)
Expression	Evaluation & conversion	—
Variants	—	Circular, Deque, Priority

Review Questions

1. Explain the **Convoy Effect** in FCFS scheduling. How does Round Robin mitigate it?
2. Given processes P1(burst=5), P2(burst=3), P3(burst=8) arriving at time 0, draw the Gantt chart for Round Robin with time quantum = 3.
3. Trace BFS on this graph starting from node **A**:

```
A -- B -- D
|    |    |
C ------- E
```

4. What is the **only** algorithmic difference between DFS and BFS?
5. How does a stack detect **unbalanced parentheses** in the expression `{[}]`?
6. Why does BFS guarantee the shortest path in an **unweighted** graph but DFS does not?
7. Name two real-world applications each for Stacks and Queues.

Looking Ahead

Next Unit: Trees

- Hierarchical data structures
- Priority Queue → Binary Heap implementation
- Tree traversals (Inorder, Preorder, Postorder)
- DFS and BFS return in the context of tree/graph algorithms!